

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250328>

Outline: Reinforcement learning in knowledge-based engineering

- A3: Reinforcement learning throughout product development lifecycle
- Elements of Reinforcement Learning (RL)
- On Knowledge-based Engineering (KBE)
- Selected examples in KBE through elements of RL
 - a. [Reinforcement Learning for Engineering Design Automation](#)
 - b. [A reinforcement learning framework for the automation of engineering decisions in complex systems](#)
- Conclusions

A3: Reinforcement learning throughout product development lifecycle

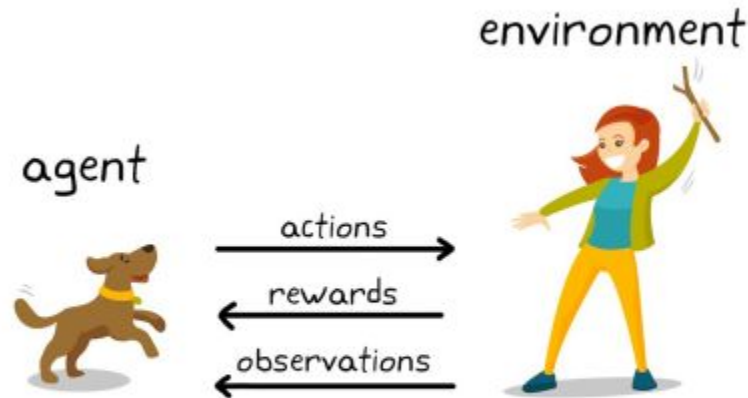
You are to investigate a potential applications of reinforcement learning (RL) throughout the product development lifecycle, focusing on 4 phases: i) product design, ii) manufacturing, iii) product use / user experience, and iv) recycling. Select at least 3 articles for each of the phases. Based on selected articles: a) Identify the main elements of RL - agent(s), environment, actions, rewards, and observations (see slide 5), b) What is an initial state for RL in selected cases? c) How the state of agent is represented and how the state space is navigated (example)? <more open questions:> d) How the exploration vs. exploitation dilemma is addressed in each of the cases? e) What APIs can be used to implement discussed approaches? - What are the most relevant / key classes for the selected APIs?

DL Fri. 11.04., 12:00 (noon)

Elements of reinforcement learning (RL)

source: Richard S. Sutton and Andrew G. Barto, “Reinforcement Learning: An Introduction”, The MIT Press, 2nd edition, 2018

Reinforcement learning (1)



Reinforcement learning in dog training. Source: Mathworks

Reinforcement learning (2)

- How to map situations to actions?
- The learner is not told which actions to take, but discovers which actions yield most reward by trying them.
- Immediate reward vs. subsequent reward.
- Exploitation vs. exploration. (Exploration-exploitation dilemma is unresolved.)
 - A distinctive challenge of RL in comparison to supervised and unsupervised learning.

RL: Elements

- Policy: Current perceived state (St_{curr}) $\rightarrow$ Action
- Reward signal: Goal: Action $\rightarrow$ Reward
- Value: $St_{curr} \rightarrow \text{Reward}(St_{curr}) + \text{Reward}(St_{curr+1}) + \text{Reward}(St_{curr+2}) + \dots + \text{Reward}(St_{curr+n})$, where n is defined by a long-term goal.
 - $\max(\text{Value}(St_{curr}))$ vs. $\max(\text{Reward}(St_{curr}))$
- Model (of the environment): $St_{curr} \rightarrow [Action_1, Action_2, \dots, Action_n]$ for $\max(\text{Value}(St_{curr}))$

RL: Elements: Policy

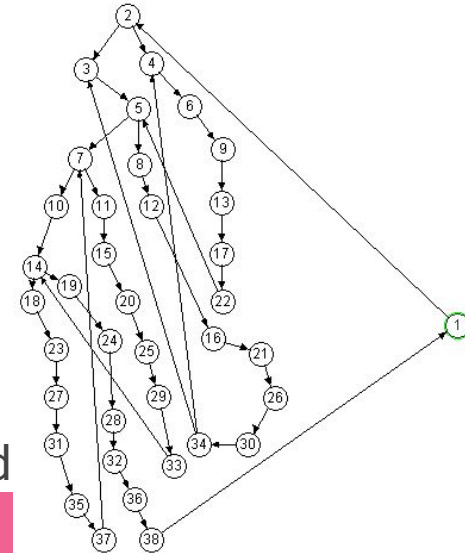
- Defines the learning agent's way of behaving at a given time.
- Maps a perceived state of the environment to actions when at a given state.
- May be a simple function or lookup table or involve extensive computations, e.g. search process.
- Sufficient to determine behaviour.
- Can be stochastic having probabilities for each action.

RL: Elements: Reward signal

- Defines a goal of a RL problem.
- On each time step / state, the RL agent receives a single number called the reward.
- The agent's sole objective is to maximise the total reward it receives over the long run.
- Reward signal defines what are good and bad events for the agent.
- It is a basis for altering the policy. If selected action by the policy leads to low reward, the policy may be changed to select some other action in that situation.
- Rewards signals may be stochastic functions of state and actions.

RL: Elements: Value function

- Specifies what is good in a long run.
- Value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
- A state may have low immediate reward, but have a high value, as higher reward states are expected to follow after that state.
(Reverse can be also true.)
- The purpose of estimating the values is to achieve more reward.
- Rewards are given by the environment; values are estimated and re-estimated from the observations.



RL: Elements: Model of the environment

- Mimics behaviour of the environment.
- Allows inferences about how the environment will behave.
- For example, for given state and action to predict next state and next action.

- Model-based methods: Solving RL problems using models and planning.
- Model-free methods: Trial-and-error learners.
- (RL spans the spectrum from low-level, trial-and-error to high-level, deliberative planning.)

Incremental implementation: action-value methods (1)

Q_k - estimation of k th reward.

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}$$

Nice, however:

Averaging methods are appropriate in a **stationary** environment, but not if rewards changing over time. (in **nonstationary** problems).

$$\begin{aligned} Q_{k+1} &= \frac{1}{k} \sum_{i=1}^k R_i \\ &= \frac{1}{k} \left(R_k + \sum_{i=1}^{k-1} R_i \right) \\ &= \frac{1}{k} (R_k + (k-1)Q_k + Q_k - Q_k) \\ &= \frac{1}{k} (R_k + kQ_k - Q_k) \\ &= Q_k + \frac{1}{k} [R_k - Q_k], \end{aligned}$$

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} [\text{Target} - \text{OldEstimate}]$$

$$\alpha = \frac{1}{k}$$

$$0 < \alpha \leq 1.$$

$$Q_{k+1} = Q_k + \alpha [R_k - Q_k]$$

Incremental implementation: action-value methods (2)

$1 - \alpha < 1$: weight of R_i decreases, decays exponentially.

If $1 - \alpha = 0$: all weight to the last reward,
as $0^0=1$.

$$Q_{k+1} = Q_k + \alpha [R_k - Q_k]$$

$$\begin{aligned} Q_{k+1} &= Q_k + \alpha [R_k - Q_k] \\ &= \alpha R_k + (1 - \alpha)Q_k \\ &= \alpha R_k + (1 - \alpha) [\alpha R_{k-1} + (1 - \alpha)Q_{k-1}] \\ &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 Q_{k-1} \\ &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 \alpha R_{k-2} + \\ &\quad \dots + (1 - \alpha)^{k-1} \alpha R_1 + (1 - \alpha)^k Q_1 \\ &= (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i. \end{aligned}$$

Knowledge-based Engineering

or Design Automation (TMM4270)

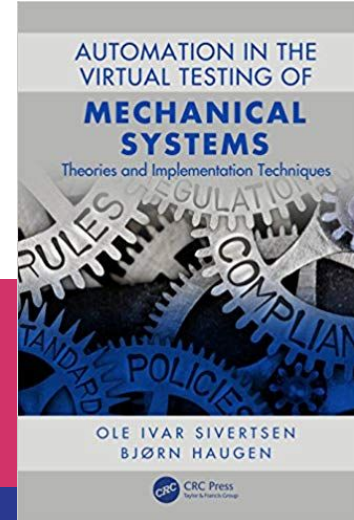
Knowledge-based Engineering (KBE) - 1

“Knowledge-based engineering (KBE) is a technology based on the use of dedicated software tools called KBE systems, which are able **to capture and systematically reuse product and process engineering knowledge, with the final goal of reducing time and costs of product development** by means of the following:

- Automation of repetitive and non-creative design tasks;
- Support of multidisciplinary design optimization in all phases of the design process.”

as (Sivertsen and Haugen, 2018)

taking into use La Rocca’s definition (2012)

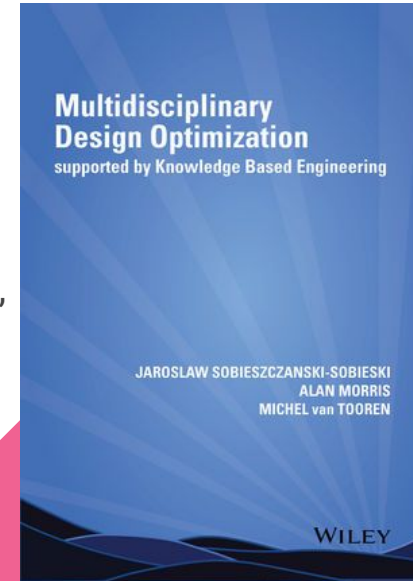


Knowledge-based Engineering (KBE) - 2

Simple definition:

“Knowledge-based engineering (KBE) is engineering using product and process knowledge that has been captured and stored in dedicated software applications, to enable its direct exploitation and reuse in the design of new products and variants.”

(Jaroslaw Sobieszczanski-Sobieski, Alan Morris, Michel van Tooren,
“Multidisciplinary Design Optimization Supported by Knowledge Based Engineering”,
Wiley, 2015)



KBE: definitions

“... KBE systems ... particular focus on their embedded programming language. This language is the core element of any KBE system and allows the capture and reuse of the design knowledge necessary to model complex engineering products.”

(Jaroslaw Sobieszczanski-Sobieski, Alan Morris, Michel van Tooren, “Multidisciplinary Design Optimization Supported by Knowledge Based Engineering”, Wiley, 2015)

Example: Ball in a groove

Coulomb Friction

▼ Friction Parameters

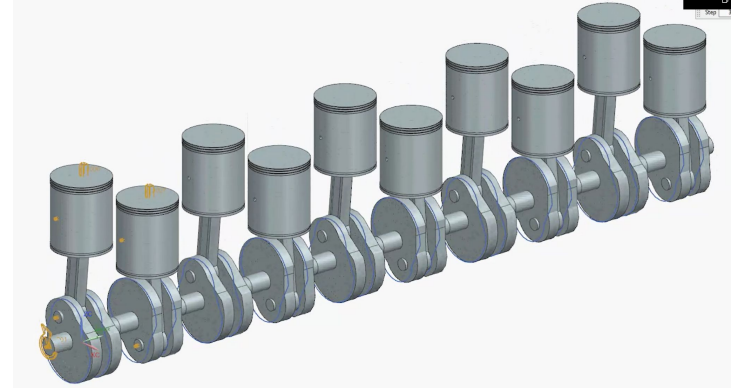
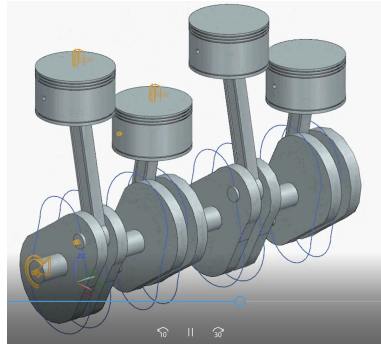
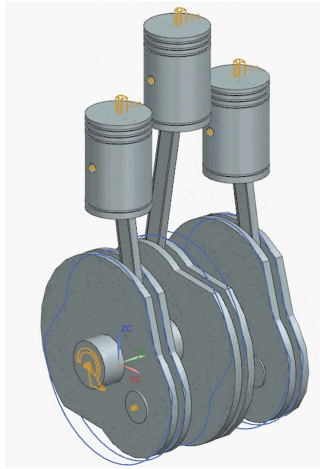
Static Coefficient	0.2
Stiction Velocity	0.01 mm/s
Dynamic Coefficient	0.1
Friction Velocity	0.05 mm/s

Time 0.0800000
Step 8



NB!
Video: **10sec** shown in **1sec**.

Example: Role of design automation (done with Python programming language)



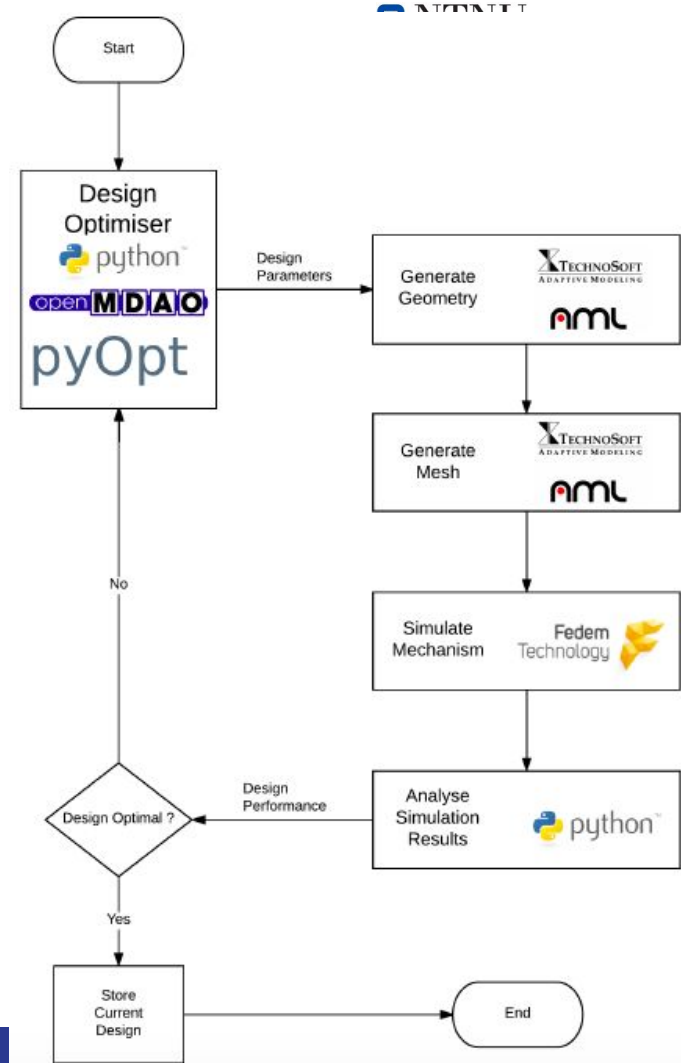
TMM4270 KBE, Intro course (2021) students work example [Mats Guddingsmo and Andreas Lindkjenn Bø]

MSc example

Optimization loop example.

From:

Arnt Underhaug Lima, “Optimizing KBE Generated Mechanisms”, Master’s Thesis, Norwegian University of Science and Technology, NTNU, 2017



Examples of RL (elements) in KBE

Reinforcement Learning for Engineering Design Automation

Fabian Dworschak, Sebastian Dietze, Maximilian Wittmann, Benjamin Schleich, Sandro Wartzack, Reinforcement Learning for Engineering Design Automation, Advanced Engineering Informatics, Volume 52, 2022, 101612, ISSN 1474-0346, <https://doi.org/10.1016/j.aei.2022.101612>.

Highlights

- Mapping of engineering requirements and parametric models into learning environments.
- Parametric CAD models as learning environments and requirements as reward functions.

Two issues for data-driven Design Automation in engineering design:

- Sparse training data resulting from differing design samples.
- Limited number of samples in the training data as consequence of short or insufficient product history.

RL Agent

“ The objective of design agents is to learn a policy, which describes a chain of agents’ decisions, leading from an initial design to a suitable design by modifying geometric parameters.”

Implemented with CAx API, Environment, Agent and DQN module.

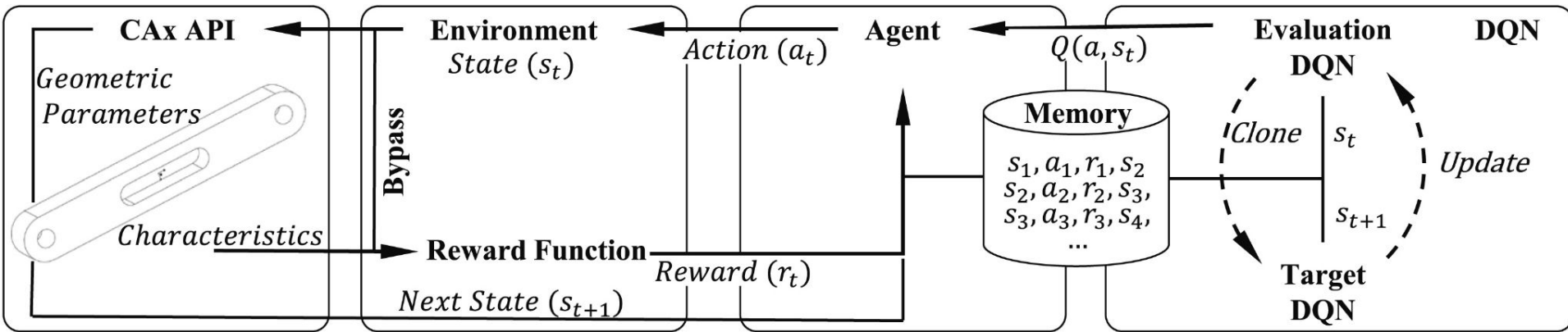
Based on Temporal-Difference (TD) method:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]. \quad (6.6)$$

$$NewEstimate \leftarrow OldEstimate + StepSize \left[Target - OldEstimate \right]. \quad (2.4)$$

Structure and relations of modules for an RL application

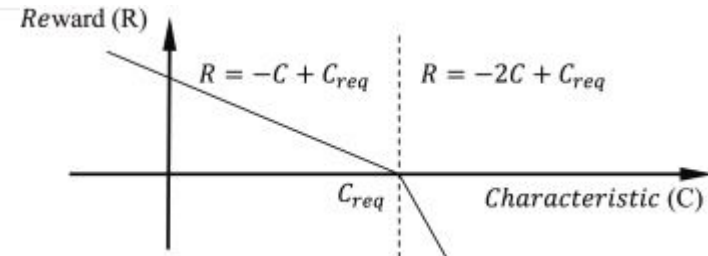
s. 46.



Environment

- Interpretation or specification of the design task
- The parameter grid defines the initial value, the step size and if necessary the boundary conditions of every adjustable geometric parameter.
- Action space 'reduce', 'keep', 'increase'.
- Reward function can be seen as an objective function of the RL agent.
Requirements: fixed, maximal, minimal or interval.

ID	initial value	lower	upper	step size
p_1	180	130	230	6
...				



API (1) source

NXOpen Python API Reference 10.0.0 documentation »

Table Of Contents

- CylinderBuilder Class
- Properties
- Methods
- Enumerations
- Property Detail
 - Arc
 - Axis
 - BooleanOption
 - Diameter
 - Direction
 - Height
 - Origin
 - ParentAssociativity
 - ReverseDirection
 - Type
- Method Detail
 - Validate

Quick search

Enter search terms or a module, class or function name.

CylinderBuilder Class

class `NXOpen.Features.CylinderBuilder`

Bases: `NXOpen.Features.FeatureBuilder`

Represents a `NXOpen.Features.Cylinder` builder

To create a new instance of this class, use `NXOpen.Features.FeatureCollection.CreateCylinderBuilder`

Default values.

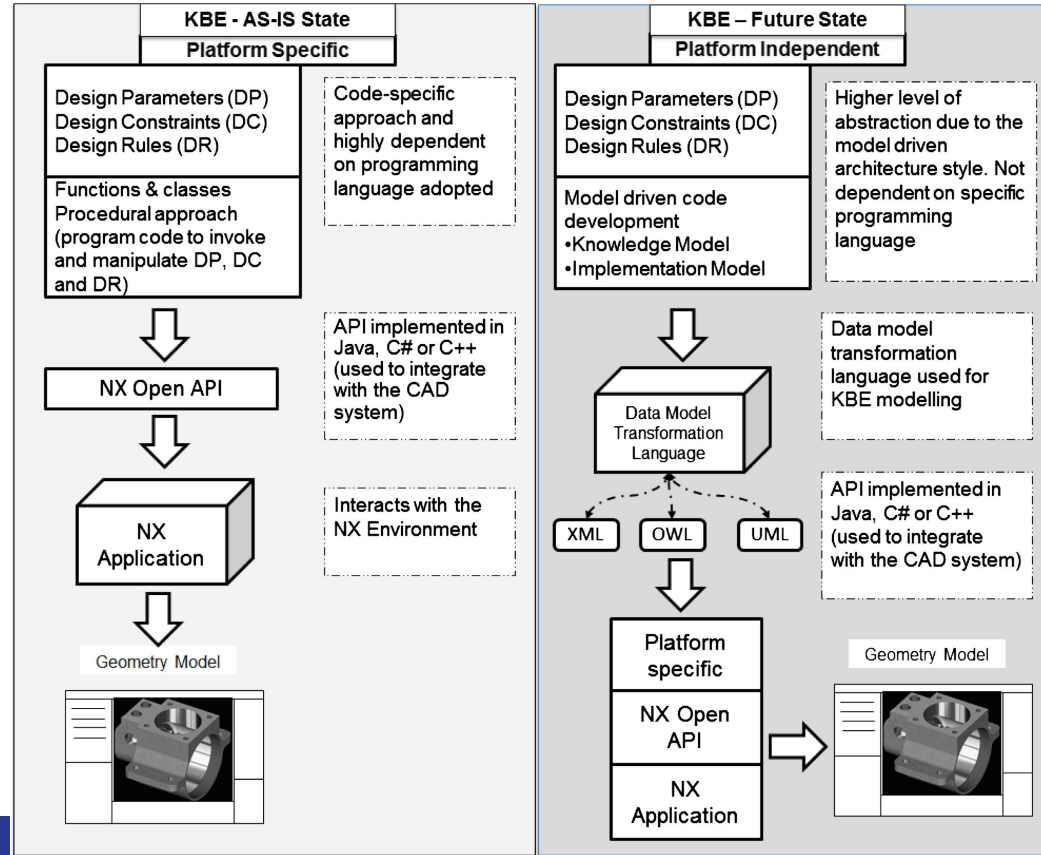
Property	Value
<code>BooleanOption.Type</code>	Create
<code>Diameter.Value</code>	50.0 (millimeters part), 2.0 (inches part)
<code>Height.Value</code>	100.0 (millimeters part), 4.0 (inches part)
<code>ReverseDirection</code>	false
<code>Type</code>	AxisDiameterAndHeight

New in version NX5.0.0.

Properties

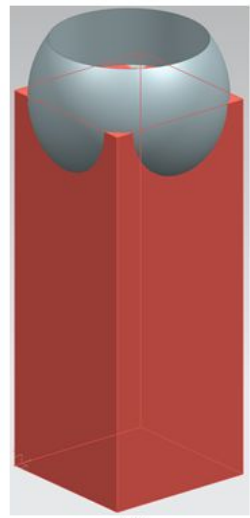
Property	Description
<code>Arc</code>	Returns the arc
<code>Axis</code>	Returns or sets the axis

API (2) source

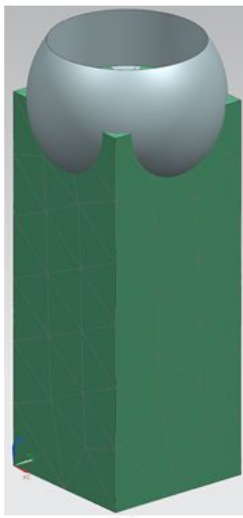


API (3) source

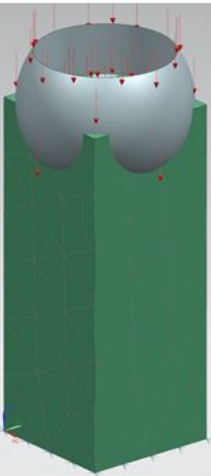
```
a = Analyzer("test_fem_", "C:\\Users\\andreil\\  
a.getSolidBodies()  
a.createFEM("8.79", "0.879", "Aluminum_5086")  
a.defineSim()  
a.assignConstrain(1, 3)  
a.assignLoad(50000, 1, 4, [0,0,-1]) # Reverse  
a.solveSim()
```



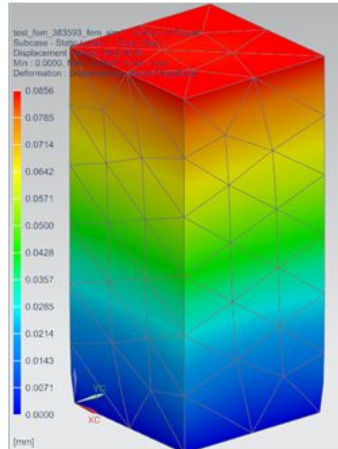
(a)



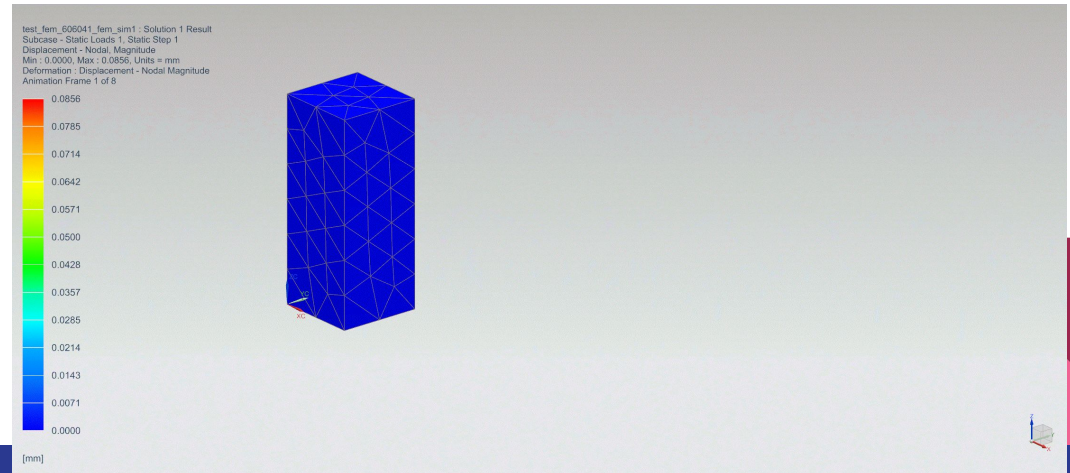
(b)



(c)



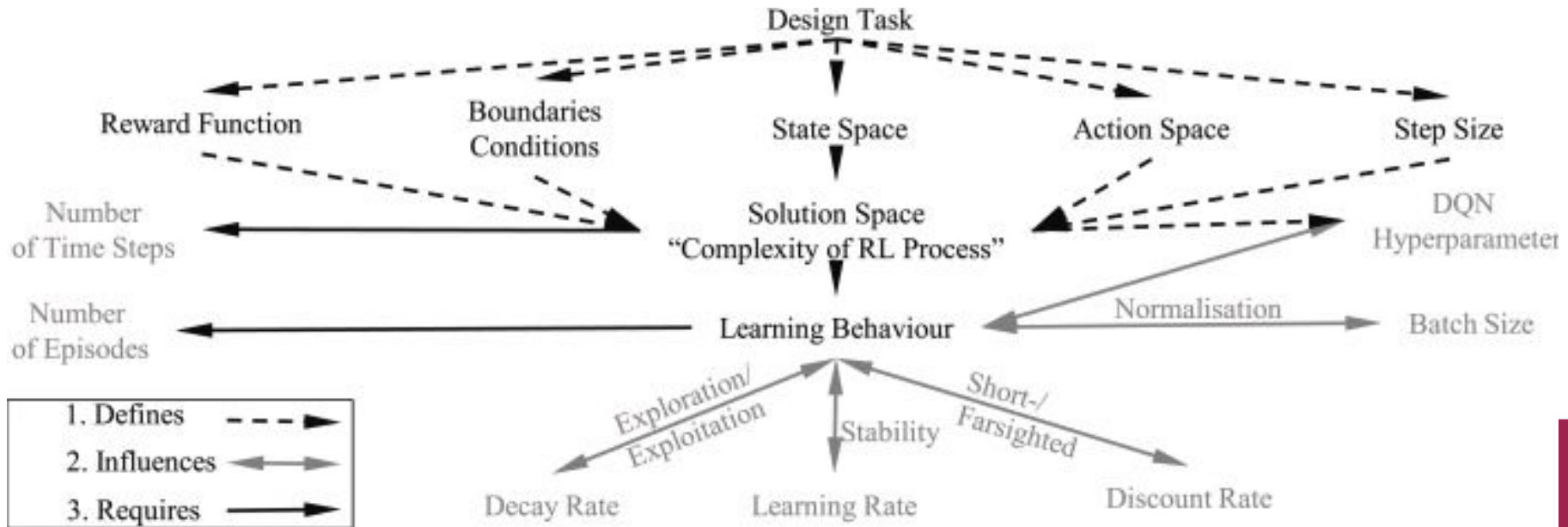
(d)



Agent

- Contains attributes for
 - Decay Rate ($0 \leq \varepsilon \leq 1$)
 - Learning Rate ($0 \leq \alpha \leq 1$)
 - Discount Factor ($0 \leq \gamma \leq 1$)
 - Number of Episodes and Time Steps
 - Batch Size and Training Start.
- γ - represents the relative importance of the future reward versus the current reward. 0 - short-sighted; 1 - farsighted decisions.

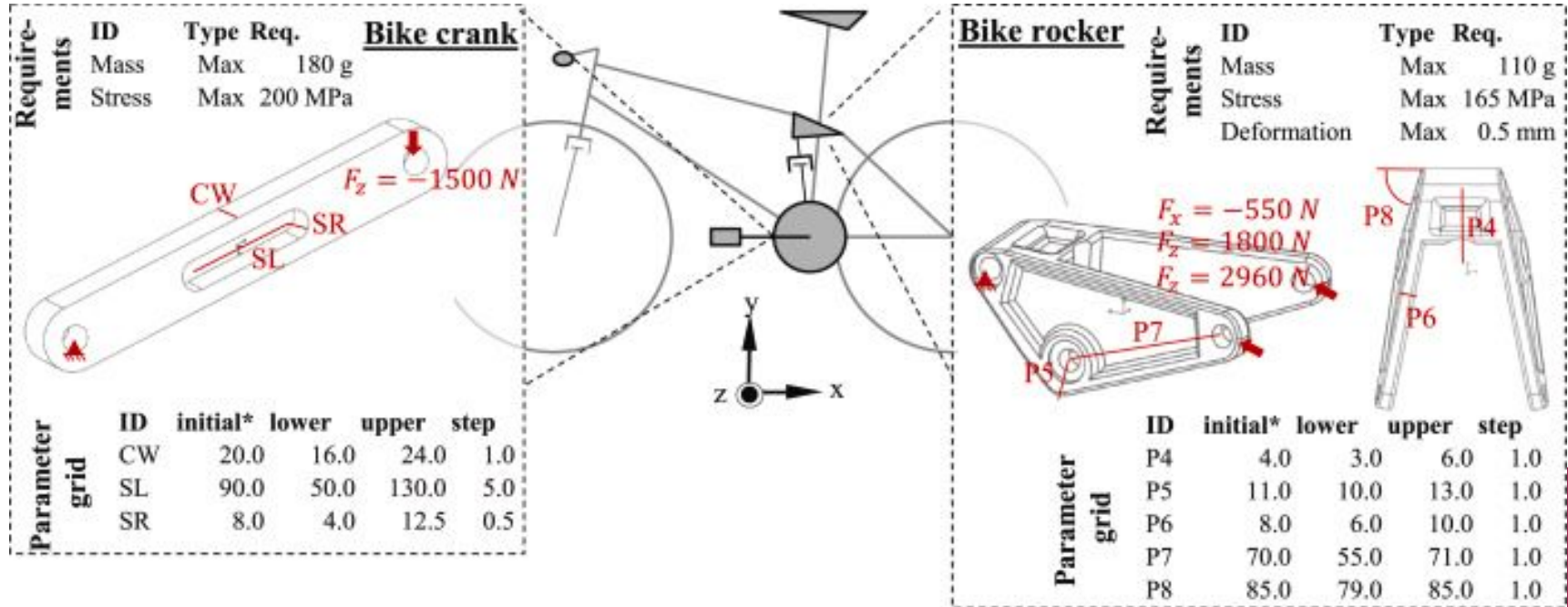
Agent: predefined (black) and adjustable (grey) parameters for design engineers



DQN: Deep Q Learning

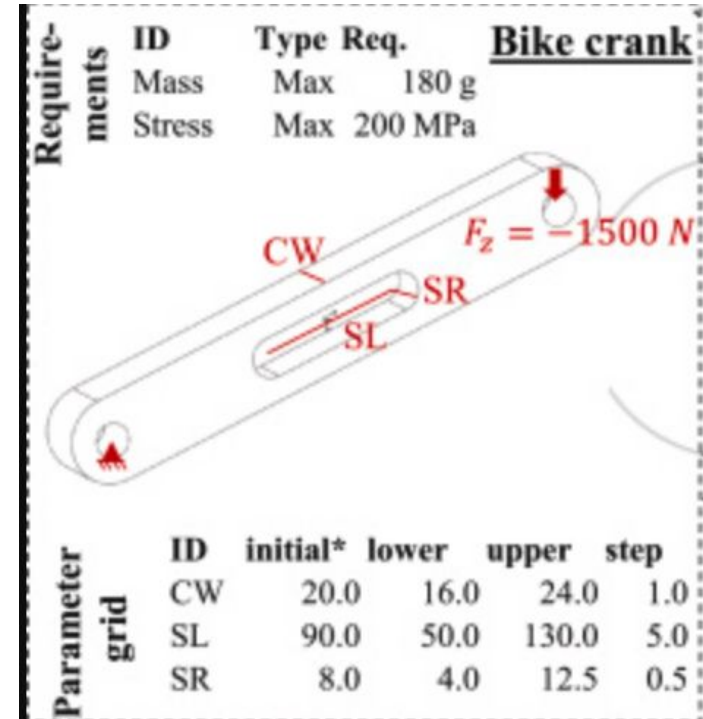
- The DNN architecture is specified in the class DQN.
- Design engineers define the number of layers and the number of neurons per layer.
- The output layer is defined by the number of state–action pairs the DQN predicts values for.

DQN: Two design tasks

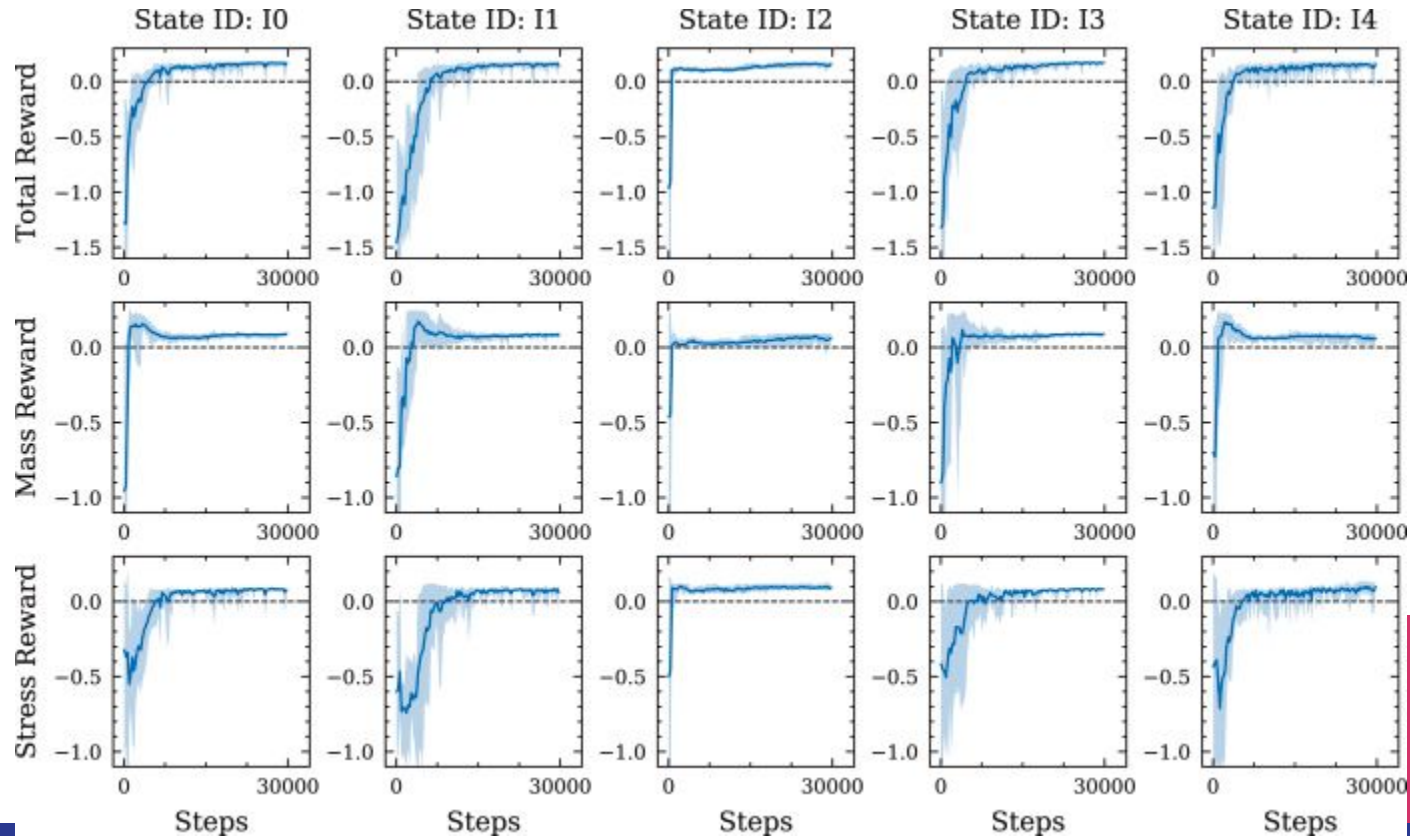


Crank and initial states (I0-I4)

- I0 [20.0, 90.0, 8.0],
- I1 [16.0, 50.0, 4.0],
- I2 [24.0, 130.0, 12.0],
- I3 [17.0, 100.0, 5.5],
- I4 [21.0, 75.0, 10.5]
- crank width (CW), slot length (SL),
slot radius (SR).

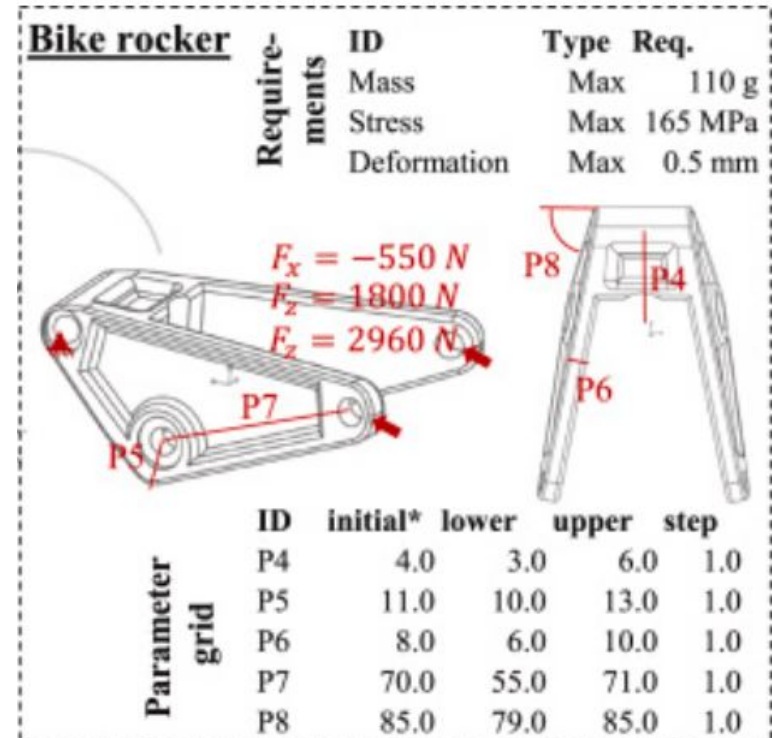


DQN: Stress, Mass, Total rewards @ states I0-I4 (crank)



Rocker and initial states (I0-I4)

- I0 [4.0, 11.0, 8.0, 70, 85.0],
- I1 [6.0, 13.0, 10.0, 71.0, 85.0],
- I2 [3.0, 10.0, 6.0, 55.0, 79.0],
- I3 [3.0, 10.0, 10.0, 71.0, 85.0],
- I4 [6.0, 10.0, 6.0, 71.0, 79.0]
- distance to seat post (P4),
distance to damper bearing (P6),
rocker width (P5),
rocker angle (P7),
rocker length (P8)



DQN: Deformation, Stress, Mass, Total rewards @ states I0-I4 (rocker)

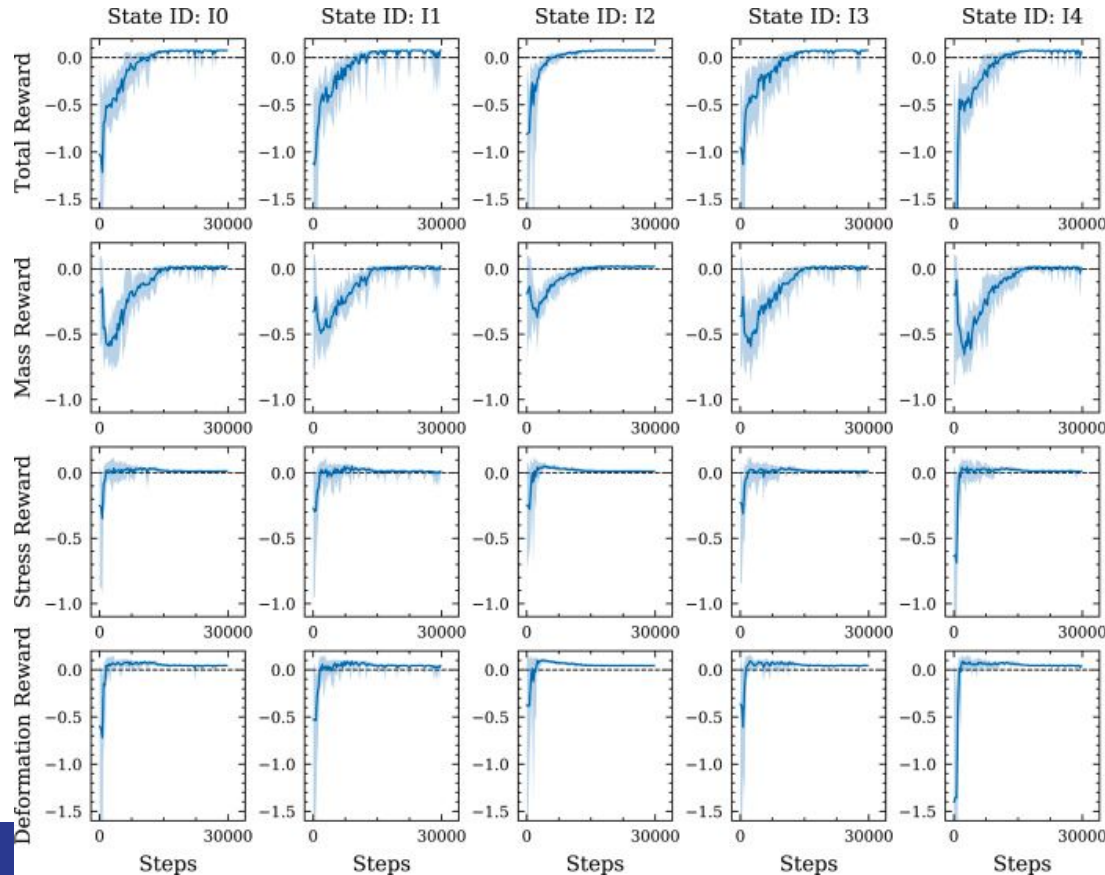
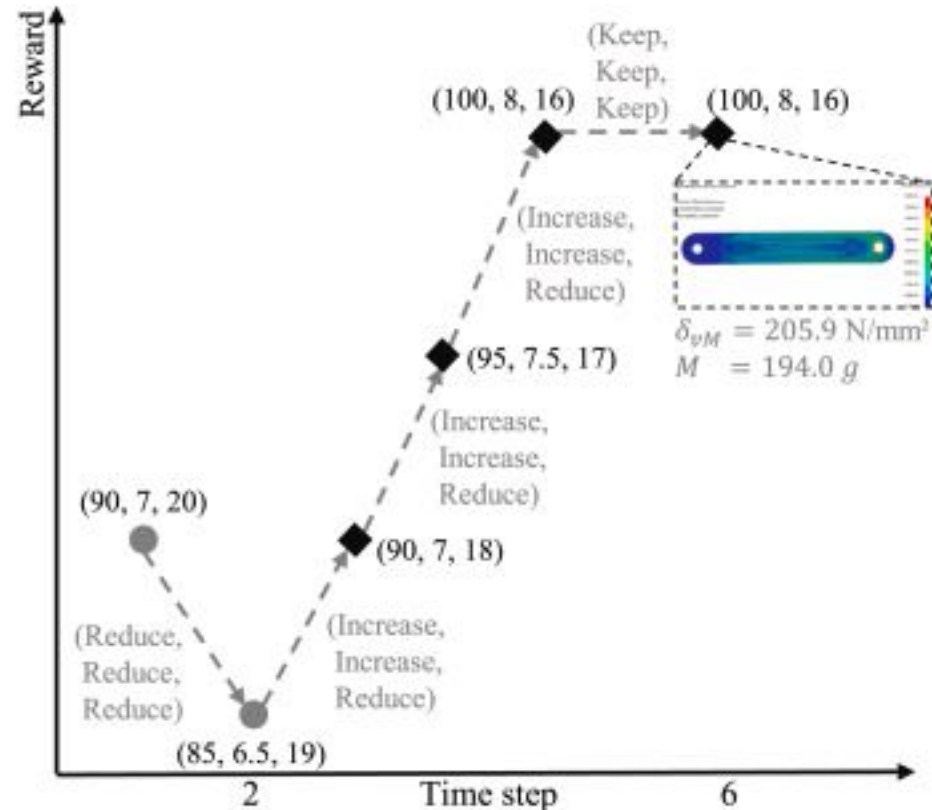
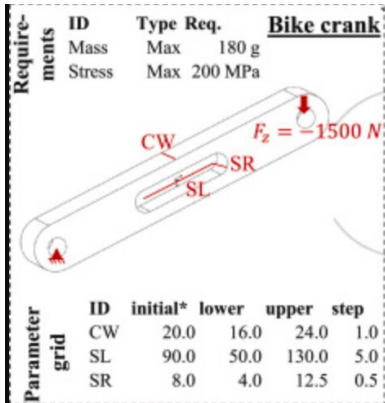


Illustration of S, R, A Explore vs. Exploit



A reinforcement learning framework for the automation of engineering decisions in complex systems

Arun Ramamurthy, Doctoral Dissertation, 2019

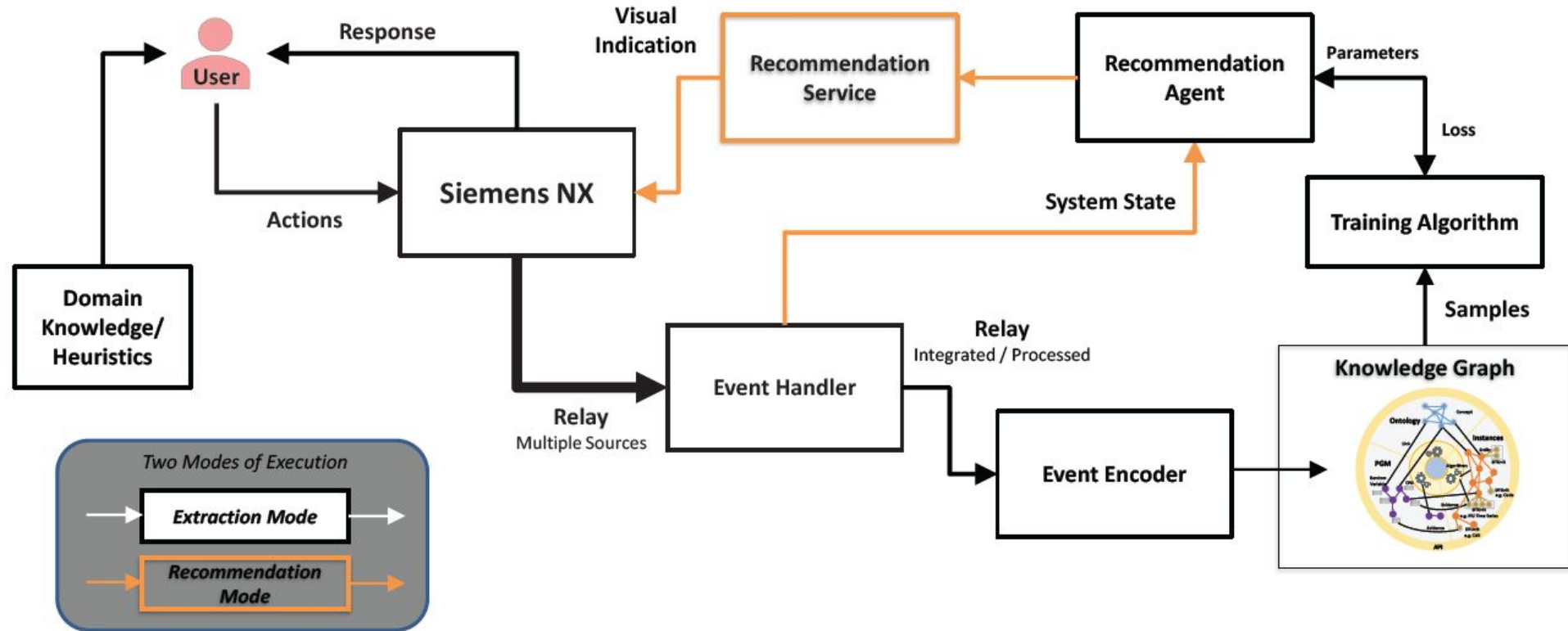
LINK: <https://smartech.gatech.edu/handle/1853/62626>

Research goal

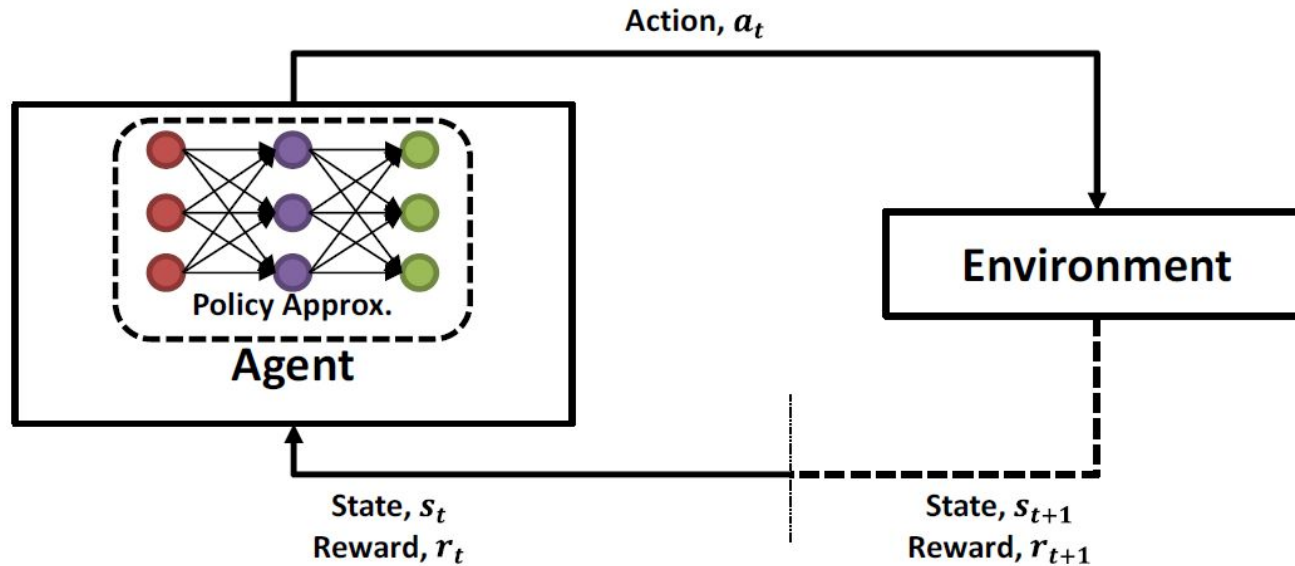
The research work aims to develop a framework that enables the extraction of knowledge from a design system such that an intelligent agent can learn to mimic the extracted behavior with an ability to self-learn in order to assist design engineers during the usage of the design system so as to develop products with shorter time to market, improved process efficiency and improved product flexibility and quality.

Design of UAV as a case study.

System architecture



Reinforcement learning



Representation of the System State

(2037, 1) dimensional vector to generate the representation of the state of the system.

Vector Source	Interpretation	Data Type	Dimension
Motor	Attributes of the motor that is on board the UAV	Real	(7, 1)
Battery	Attributes of the battery that is on board the UAV	Real	(7, 1)
Propeller	Attributes of the propeller driven by the motor	Real	(6, 1)
ESC	Attributes associated with the electronic speed controller	Real	(5, 1)

Servo	Attributes of the servo motor onboard the UAV	Real	(4, 1)
Payload	Vector representing the presence of a certain type of payload	Integer $\in [0, 1]$	(5, 1)
Architecture	Index of the active architecture	Integer $\in [-1, 3]$	(1, 1)
Machine Id	Index of the 3D printer used for manufacturing	Integer $\in [-1, 2]$	(1, 1)
Scalable Parameter	Represents the value of the scalable parameter associated with the architecture	Real	(1, 1)
Requirements	Embedding of the requirements set	Real	(20, 100)

Representation of Actions (1)

- Encoded in two ways - for discrete choices and continuous parameters.
- Actions (discrete parameters) represented as a one-hot vector in which each vector entity represents a choice of a component or architecture.
- Actions continuous scalable parameter, a single real number is utilized to represent the value of the parameter within predefined bounds.

Scenario Id	Battery	Motor	Propeller	ESC	Servo	Payload	Architecture	Machines
1	10	11	6	5	5	5	4	3
2	50	25	15	25	25	5	4	3

Representation of Actions (2)

Scalable Parameter Type	Parameter Bounds
Fixed Wing Vehicle: Wing Area	$0.05 - 0.70 \text{ m}^2$
Multicopter Vehicle: Arm Length	$5 - 12 \text{ inches}$

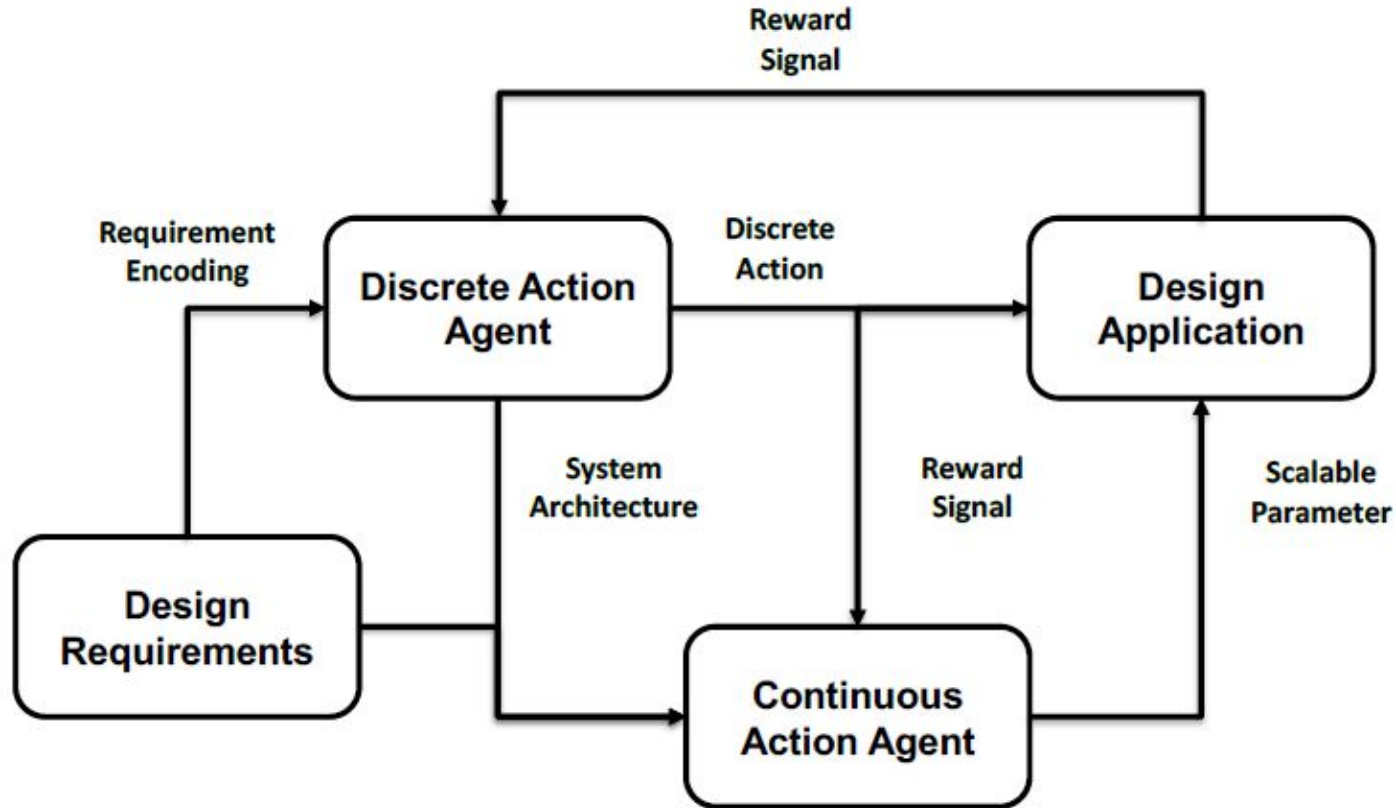
Reward formulation (2)

$$\text{Reward} = f(\text{State}, \text{Action})$$

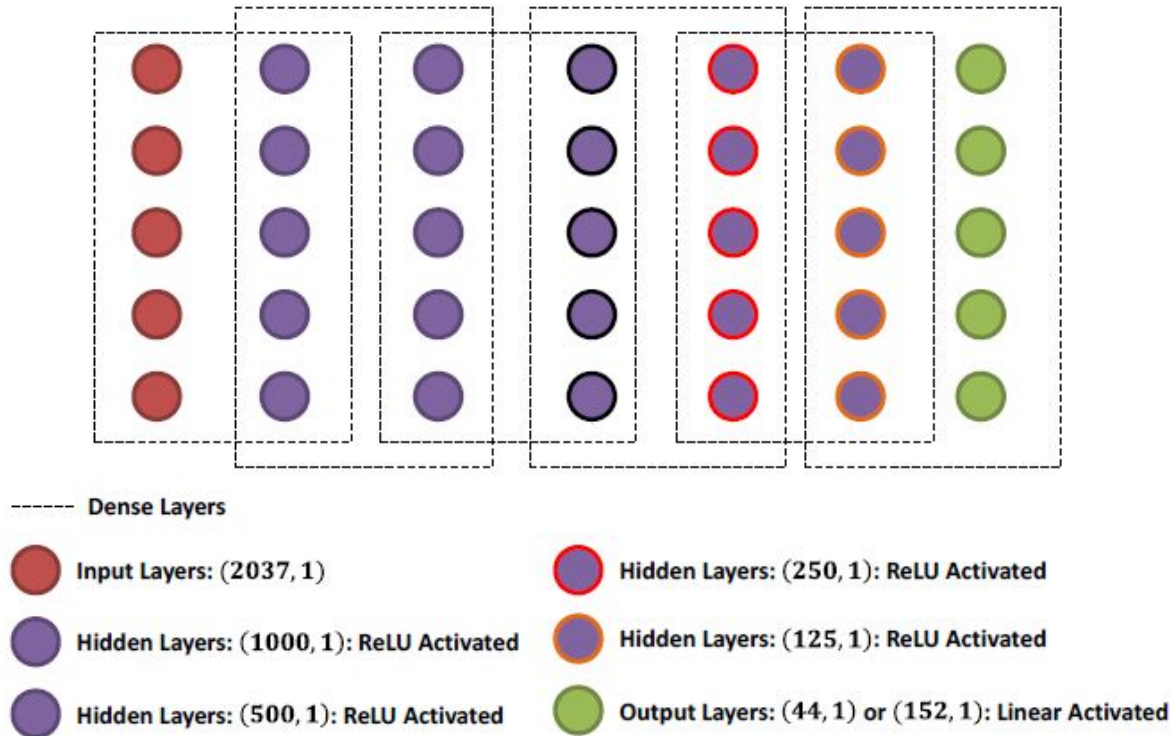
$$= \begin{cases} -10, & \text{State is not compatible} \\ -1, & \text{State is compatible, but design is incomplete} \\ -5, & \text{State is compatible and complete,} \\ & \text{but requirements are not satisfied} \\ g(w_{tot}, t_{manf}) > 0, & \text{State is compatible and complete and} \\ & \text{the requirements are satisfied} \end{cases}$$

Agent architecture

Compare to [slide 24](#).



Double DQN agent to predict or recommend discrete architectural parameters



ReLU

Conclusions

- Two examples of Design Automation or KBE seen through the *elements* of the Reinforcement Learning.
- Ad hoc, specific solutions for representing States, Actions, Rewards and Agent(s).
 - Design parameters: fixed, max, min, interval / discrete, continuous.
- APIs allow connection to the CAD tools to demonstrate and analyse (e.g. FEM) a product. CAD is used as an environment to calculate reward signals.

References

1. Richard S. Sutton and Andrew G. Barto, “Reinforcement Learning: An Introduction”, The MIT Press, 2nd edition, 2018
2. Fabian Dworschak, Sebastian Dietze, Maximilian Wittmann, Benjamin Schleich, Sandro Wartzack, Reinforcement Learning for Engineering Design Automation, Advanced Engineering Informatics, Volume 52, 2022, 101612, ISSN 1474-0346, <https://doi.org/10.1016/j.aei.2022.101612>
3. Arun Ramamurthy, “Login SMARTech Home Georgia Tech Theses and Dissertations Georgia Tech Theses and Dissertations View Item A reinforcement learning framework for the automation of engineering decisions in complex systems”, Georgia Institute of Technology, Doctoral Dissertation, 2019, Available online: <https://smartech.gatech.edu/handle/1853/62626>